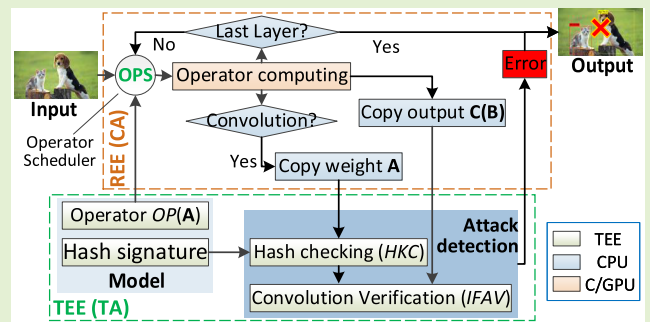


Verifiable Deep Learning Inference on Heterogeneous Edge Devices With Trusted Execution Environment

Longlong Liao^{ID}, Yuqiang Zheng^{ID}, Hong Lu^{ID}, Xinqi Liu, Shuguang Chen^{ID}, and Yuanlong Yu^{ID}

Abstract—Deep learning inference on edge devices is susceptible to security threats, particularly fault injection attacks (FIAs), which are easily executed and pose a significant risk to the inference. These attacks could potentially lead to alterations to the memory of the edge device or errors in the execution of instructions. Specifically, time-intensive convolution computation is considerably vulnerable in deep learning inference at the edge. To detect and defend attacks against deep learning inference on heterogeneous edge devices, we propose an efficient hardware-based solution for verifiable model inference named DarkneTV. It leverages an asynchronous mechanism to conduct hash checking of convolution weights and verification of convolution computations within the trusted execution environment (TEE) of the central processing unit (CPU) when the integrated graphics processing unit (GPU) runs model inference. It protects the integrity of convolution weights and the correctness of inference results, and effectively detects abnormal weight modifications and incorrect inference results regarding neural operators. Extensive experimental results show that DarkneTV identifies tiny FIAs against convolution weights and computation with over 99.03% accuracy but less extra time overhead. The asynchronous mechanism significantly improves the performance of verifiable inference. Typically, the speedups of the GPU-accelerated verifiable inference on the Hikey 960 achieve $8.50 \times -11.31 \times$ compared with the CPU-only mode.

Index Terms—Deep learning inference, fault injection attacks, heterogeneous edge devices, trusted execution environment (TEE), verifiable learning.



I. INTRODUCTION

EDGE intelligence processes unstructured sensitive data in several safety-critical and privacy-sensitive scenarios,

Manuscript received 18 June 2024; revised 7 July 2024; accepted 8 July 2024. Date of publication 17 July 2024; date of current version 1 September 2024. The work was supported in part by the Natural Science Foundation of Fujian Province of China under Grant 2021J01617, in part by the Program of National Natural Science Foundation of China under Grant U21A20471, in part by the Qing Lan Project of Jiangsu Province, and in part by the Hong Kong Quantum AI Lab, AIR@InnoHK of the Hong Kong Government. The associate editor coordinating the review of this article and approving it for publication was Dr. Zhenghua Chen. (Corresponding author: Yuqiang Zheng.)

Longlong Liao is with the College of Computer and Data Science, Fuzhou University, Fuzhou 350108, China, and also with the Faculty of Science, The University of Hong Kong, Hong Kong 999077, China (e-mail: liaoll@hku.hk).

Yuqiang Zheng and Yuanlong Yu are with the College of Computer and Data Science, Fuzhou University, Fuzhou 350108, China (e-mail: 221027190@fzu.edu.cn; yu.yuanlong@fzu.edu.cn).

Hong Lu is with the School of Mathematics and Information Science, Nanjing Normal University of Special Education, Nanjing 210038, China, and also with the College of Computer and Artificial Intelligence, Nanjing University of Science and Technology Zijin College, Nanjing 210094, China (e-mail: dg1633011@smail.njtu.edu.cn).

Xinqi Liu is with the Faculty of Engineering, The University of Hong Kong, Hong Kong 999077, China (e-mail: xinqi_liu28@connect.hku.hk).

Shuguang Chen is with the Faculty of Science, The University of Hong Kong, Hong Kong 999077, China (e-mail: sgchen@hku.hk).

Digital Object Identifier 10.1109/JSEN.2024.3426533

e.g., autonomous driving [1], [2], [3] and biometric recognition (including face recognition and voiceprint detection). Despite edge intelligence showing numerous advantages and potential, its reliable protection is a crucial yet under-researched challenge [4]. Edge devices feature limited computation capabilities and diverse hardware architectures, and their operating systems (OSs) are typically customized and simplified. Consequently, edge devices deployed by edge intelligence are vulnerable to attackers aiming to hijack computing operations and extract sensitive data. In typical application scenarios, edge devices are generally deployed close to users and data, which means they are scattered and cannot be specially maintained. Unfortunately, these vulnerabilities create a substantial risk of fault injection attacks (FIAs) [5]. As a result, FIAs can modify the runtime and results of deep learning inference at the edge, posing threats to edge intelligent systems. Taking a real-world accident as an example, the FIA on Ukraine's power grid resulted in a massive power outage affecting a quarter of a million people [6]. Similarly, smart grids that use edge intelligent systems are at risk of FIAs and pose significant security threats to human lives and property [7].

The convolution layer is a core operator in classic deep learning models, particularly for its applications in computer vision areas. We assess the convolution time and the total

time of model inference using Darknet [8] and calculate the proportion of the convolution time within the model inference. As examples of two classic deep learning models, Resnet18 [9] and YOLOv4-tiny [10], convolutions account for 94.71%–93.52% of model inference time, respectively. Therefore, the convolutions are the most time-consuming operators in deep learning models. The papers [11], [12], [13] present that adversaries require several prerequisites to successfully execute FIAs on deep learning inference, such as sufficient time to complete their attack process. For example, Rowhammer attacks require repeatedly accessing the memory thousands of times to change the memory data of adjacent rows. From the perspectives of cost and attack time window of FIAs, falsifying the memory used by the convolution layers is the most effective way. In addition, the model weight is privacy-aware information and is vital for obtaining reliable output in deep learning inference. It is more common for the model weights to be deployed on edge devices for long-term model inference. Conversely, each layer's input feature is an immediate and temporary variable responsible for storing the previous layer's output and input to the current layer. Compared with the model weights, the time window and probability of successfully executing an attack against input features are considerably small. Therefore, this work focuses on protecting the correctness of vulnerable convolution computations and the integrity of their weights to ensure the reliability of deep learning inference on heterogeneous edge devices.

The trusted execution environment (TEE) provides a hardware-level secure code execution environment isolated from the remaining untrusted system's software and OS in the rich execution environment (REE) [14]. It provides protection for deep learning inference from FIAs and ensures the integrity of deep learning models along with the reliability of inference results on devices equipped with TEE. However, the performance of secure code execution within the TEE is constrained by the computing capability of the edge device's central processing unit (CPU).

The integrated graphics processing unit (GPU) in the heterogeneous edge device does not support TEE but could provide more powerful computation capability for deep learning inference. For instance, the heterogeneous device Hikey 960 contains a Bifrost architecture GPU Mali-G71 MP8, whose theoretical computing capability is several times the CPU. Accelerating inference with the powerful GPU can significantly improve the performance of model inference. Therefore, it is an under-researched challenge that simultaneously leverages the CPU TEE to protect deep learning inference from FIAs and uses the integrated powerful GPU to accelerate the inference process. Open computing language (OpenCL) [15] is a framework for writing programs that execute across heterogeneous platforms consisting of CPUs, GPUs, etc. We propose DarkneTV, a verifiable and efficient deep learning solution for heterogeneous edge devices with the TEEs. DarkneTV accelerates deep learning inference with the OpenCL-enabled GPUs on heterogeneous edge devices. At the same time, the OP-TEE framework enables heterogeneous edge devices to perform TEE-based model encryption and verifiable inference. The proposed DarkneTV is designed for

various heterogeneous edge devices that consist of CPUs with TEE and GPUs with OpenCL support.

The contributions of this article are listed as follows.

- 1) DarkneTV is a practically efficient hardware-based solution for verifiable deep learning inference on heterogeneous edge devices. It protects the integrity of model weights by encrypting them with the TEE secure storage service and generating their hash signatures in the TEE before starting deep learning inference. In addition, DarkneTV takes advantage of hash checking of convolution weights and verification of convolution results based on Freivalds' algorithm to detect FIAs. It deploys the FIA detection process to the CPU TEE rather than the REE to enhance the reliability of convolution verification.
- 2) DarkneTV adopts an asynchronous mechanism to conduct convolutions on the integrated GPU, whereas the hash checking and convolution verification run in the CPU TEE. This mechanism enables the CPU–GPU cooperative execution to effectively reduce the extra time taken by the attack detection and significantly improve the verifiable inference speed on heterogeneous edge devices.
- 3) DarkneTV is evaluated using four classic deep learning models and the Hikey 960. The experimental results suggest that DarkneTV effectively detects slight attacks or perturbations on the convolution weight and convolution computation with a more than 99.03% probability. The speed of GPU-accelerated verifiable inference is improved by $8.50\times$ – $11.31\times$ compared with the CPU-only mode.

The remainder of this article is organized as follows. Section II discusses the related work. Section III presents the detailed design of DarkneTV and analyzes the differences from the previous works. The verification algorithm of convolution computations and its implementation in the TEE are described in Section IV. Then, Section V describes the threat model of FIAs and provides a simulated FIA algorithm that DarkneTV aims to defend against. Section VI illustrates the extensive experiments that are conducted to evaluate the attack detection capability and inference performance of DarkneTV. Finally, the conclusion of this work is presented in Section VII.

II. RELATED WORK

A. FIAs for Deep Learning

There are several methods to execute FIAs for deep learning inference at the edge. Classic implementations encompass Rowhammer [16], laser irradiation [17], electromagnetic pulse [18], and other methods. Rowhammer attack is a form of software attack. It can cause some bits in adjacent rows to flip by repeatedly accessing a row in the dynamic random access memory (DRAM) of devices. This action can modify the memory of running deep learning inference, leading to the failure inference result. Laser irradiation and electromagnetic pulse attacks are physical attacks. The laser irradiation attack irradiates DRAM and causes the stored data to change [19], whereas irradiating an unencapsulated CPU can cause some

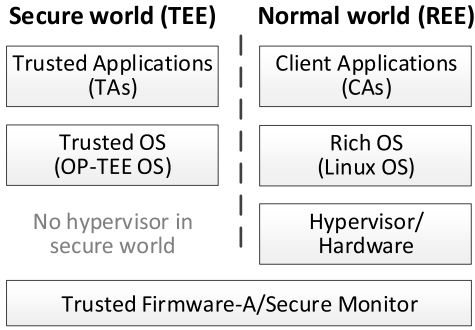


Fig. 1. ARM TrustZone technical architecture.

instructions to be skipped [20]. This significantly disrupts the process of model inference.

In deep learning inference on edge devices, FIAs typically manipulate inference results by maliciously altering model weights in memory. Several solutions leverage algorithms to select optimal attack positions for minimizing parameter modifications [11], [12], [13]. Zhao et al. [12] apply the ADMM algorithm [21] to minimize parameter modification to conceal attacks. Similarly, Bai et al. [13] apply the ADMM algorithm to determine the crucial position of Rowhammer to achieve the purpose of stealth during the process of model inference. Liu et al. [11] use gradient descent to modify biases or weights of linear layers to reduce the model's performance or modify the model inference output according to the adversaries' will. These methods enable stealthy attacks, allowing precise modifications of inference results while preserving specific input patterns. Considering that actual FIAs cannot arbitrarily affect all memory, Yao et al. [22] target reducing model accuracy to random guessing levels, prioritizing the disruption of model functionality over concealing the attacks.

B. ARM TrustZone

ARM TrustZone is a system-wide approach to security that provides hardware-assisted isolation for secure and nonsecure areas of a processor [23]. As shown in Fig. 1, it splits hardware and software in the processor into two worlds: a normal world and a secure world [24].

The normal world is used to run a rich Linux OS and offers an REE for executing nonsecure client applications (CAs). Meanwhile, the secure world uses a trusted small OS (e.g., OP-TEE OS [25]) to establish a TEE for executing trusted applications (TAs). The TA is executed within the TEE, using its secure resources to prevent and ignore threats from the REE. The hardware-enforced isolation within the TEE guarantees that the REE has no access to the TEE's memory and resources. The CA uses the TEE client API to access facilities provided by TAs inside the TEE. The TEE client API enables the CA and the TA to communicate large amounts of data quickly and efficiently via a shared memory area accessible to both the TEE and the REE [24]. The two worlds can switch to each other under strict supervision of a secure monitor running in monitor mode, which is implemented with the Trusted Firmware-A [26]. The TEE only requires additional operations to build the TA and the communication between the TA and

the CA, so its performance is comparable to normal executions in a nonsecure environment (e.g., a general OS) [27].

The ARM TrustZone technology is adopted in a wide range of edge devices such as Hikey and Hikey 960. In these devices, the TEE is implemented in the CPU with OP-TEE [25] and offers secure memory, storage, and communication between the TEE and REE. OP-TEE is an open portable TEE framework designed as a companion to a nonsecure Linux kernel running on ARM platforms. It relies on the ARM TrustZone technology as the underlying hardware isolation mechanism. Edge devices with TEEs could provide hardware-level isolation and an independent and secure code execution environment for critical edge intelligence applications.

C. Deep Learning Inference With TEE

TEE can be used to protect the privacy of deep learning models and their inference from adversaries. Mo et al. [27] propose the DarkneTZ framework to defend against member inference attacks. DarkneTZ divides a deep learning model into privacy-sensitive layers and nonprivacy-sensitive ones. The former runs in the secure world of TEE, whereas the latter runs in the normal world of REE. Shen et al. [28] propose the morph and then restore protocol, which transforms partial model operators into parameter-morphed, uses the GPU to conduct their inference, and then restores the computation results in the TEE. OMG [29] enables privacy-preserving deep learning on ARM-based edge devices by leveraging the strict hardware-enforced isolation to run model layers in the TEE. In addition, Wang et al. [30] propose Branchy-TEE to dynamically load neural networks to TEE on demand, aiming to protect the confidentiality and integrity of data and models. These approaches protect the privacy and secrecy of deep learning models and their inference but face the huge latency arising from edge devices' limited CPU computing capability or model operator transformation.

III. DESIGN OF DARKNETV

The name DarkneTV is meticulously designed. "DarkneTV" means that it is based on the TEE and the deep learning framework Darknet [8], where the uppercases "T" and "V" represent "TEE" and "Verifiable," respectively. DarkneTV consists of two stages: model encryption and verifiable inference. The first stage encrypts model weights and generates hash signatures of convolution weights in the TEE to protect the integrity and privacy of deep learning models on the edge device. The second stage detects FIAs against the model inference process and aims to verify the dependability of output results. It contains hash checking of convolution weight signatures and verification of convolution computation results, and both are performed in the TEE. Besides, it offers deep learning inference with either the CPU or the integrated GPU on the heterogeneous edge device.

For the sake of expression, in this article, the weight parameters of one operator (layer) are denoted as the matrix **A**; the input feature map, namely, the output of the previous layer, is denoted as the matrix **B**; the output feature map is denoted as the matrix **C**. HA_1 denotes the hash signature of

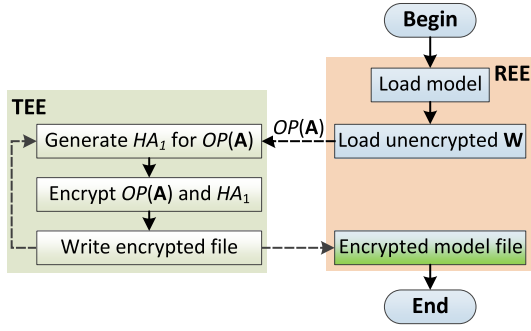


Fig. 2. Model encryption in DarkneTV.

A generated with the SHA-256 algorithm [31] in the TEE before executing a convolution operator. HA_2 denotes the hash signature of A generated with the SHA-256 algorithm in the TEE for detecting attacks on A after completing the convolution computation. $OP(A)$ means the computation of a neural operator in deep learning inference based on the weight A , such as convolution, activation, and pooling. KHC denotes checking the two hash signatures of A (HA_1 and HA_2), whereas IFAV denotes verifying a convolution computation with Freivalds' algorithm. The combination of KHC and IFAV refers to the attack detection procedure of convolutions in DarkneTV. **OPS** denotes an operator scheduler that receives and launches inference instructions of model operators.

A. Model Encryption

DarkneTV introduces the model encryption shown in Fig. 2 to ensure the integrity and privacy of the model structure and weights stored at the edge device. This stage is performed offline in a relatively secure environment before starting deep learning inference on edge devices. First, DarkneTV loads the model structure and trained model weight W in the REE. Second, it transfers all the operators $OP(A)$ in the deep learning model from the REE to the TEE and then generates the hash signature HA_1 for each convolution weight with the SHA-256 algorithm in the TEE. Third, the invoked TA instance calls the TEE file system to encrypt HA_1 and model operators for writing encrypted data. Finally, DarkneTV stores the encrypted operators and hash signatures of convolution weights as persistent files in the Linux file system in the REE. These encrypted model files can only be read and decrypted with the TEE in the current CPU since the decrypt key is stored in the TEE of the current device.

B. Verifiable Inference

1) **Overview:** The second stage performs verifiable inference on heterogeneous edge devices with TEE and aims to obtain correct model inference results. It could be accelerated with an integrated GPU to improve the inference performance. Fig. 3 shows the verifiable inference implemented in the proposed DarkneTV, depicting the components of verifiable inference and their handling in the face of FIAs. The light green processes denote running in the TEE with a CPU core, while the light blue ones represent running on the CPU in the REE. The orange processes represent that these computation tasks

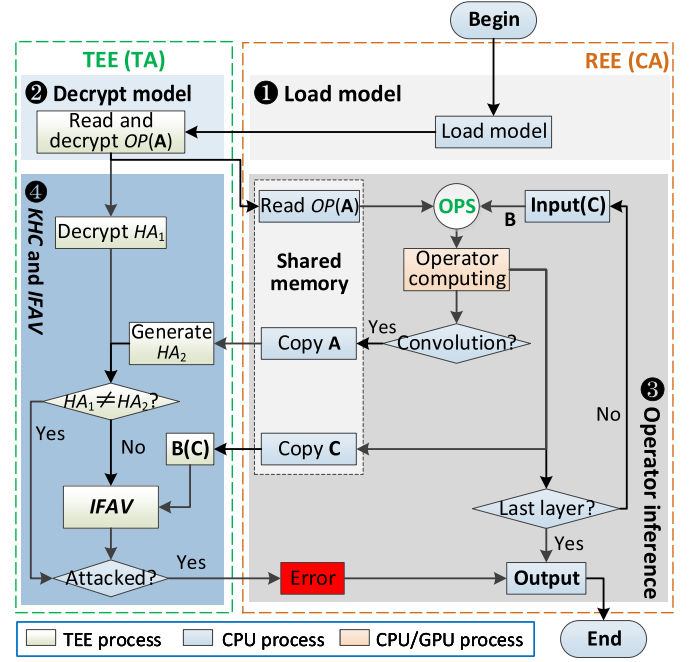


Fig. 3. Verifiable inference in DarkneTV. (**OPS** denotes an operator scheduler.)

can be offloaded to and run on either the CPU or the integrated GPU in the REE.

The verifiable inference in DarkneTV consists of four procedures: ① load the model structure and initialize its hyperparameters to build a deep learning model instance in the REE; ② read and decrypt model operators in the TEE; ③ execute operator inference in the REE with the CPU/GPU; and ④ conduct the attack detection (KHC and IFAV) in the TEE. DarkneTV creates a CA and TA instance to iteratively execute the above four procedures under the scheduling of the operator scheduler **OPS** to implement the verifiable inference on heterogeneous edge devices.

2) **Operator Inference:** When starting the verifiable inference procedure, the CA creates a thread pool and opens a TEE session. The thread pool aims to reduce the time overhead of creating and destroying threads by reusing them. The TEE session offers secure data communication between the CA and TA instance. First, ① the CA invokes the TA instance to load the model. Second, ② the TA instance reads and decrypts the model weights and hash signatures encrypted files that are generated in the stage of model encryption. The TA instance transfers the decrypted weights to the CA, whereas the decrypted hash signatures HA_1 are kept in the TA's secure memory for subsequent KHC. Third, ③ the CA reads the decrypted operator $OP(A)$ and the input B to execute the operator computation. Input B comes from the input data (e.g., image, video) of the first operator or its previous operator output C . Thus, C is simultaneously transferred to the TA instance as the computed result and the next operator's secure input B . When the current operator is the first one and a convolution layer, the CA transmits B to the TA instance to execute the IFAV. The continuous operators' inference processes run with the main thread, while the KHC and IFAV ④ processes run with the idle thread from the thread pool.

3) Kernel Checking: During operator computing in the REE on the CPU/GPU, malicious attacks may modify the preload operator $OP(A)$. That is especially true for time-consuming convolution computations. The KHC is designed to detect attacks against the convolution $OP(A)$ in the REE when executing convolution inference on the CPUs or the integrated GPUs of edge devices.

When completing a convolution inference, the main thread submits a task to the thread pool, aiming to transfer the used convolution weight A to the TA instance. The TA instance generates the hash signature HA_2 with the SHA-256 algorithm for the received A . After that, the TA instance compares HA_2 with HA_1 to conduct the KHC, as shown in Fig. 3. The two hash signatures, HA_1 and HA_2 , are different, demonstrating that the attackers modify A in the REE during this convolution inference on the CPU/GPU. The TA instance sends a message that this inference case is attacked, and its corresponding convolution output C is incorrect. In this case, DarkneTV reports a found attack and will not verify the convolution output C with the IFAV again. Otherwise, the case that they are the same suggests that A used by the inference is not attacked. It is necessary to execute the IFAV in Fig. 3 to verify the correctness of the convolution output C .

4) Convolution Verification: If the CA completes a convolution computation and the KHC procedure does not detect the attack on its weight A ($HA_1 = HA_2$), the main thread submits an IFAV task to the thread pool. Then, an idle thread in the pool invokes the TA instance to execute this IFAV task in the TEE. The TA instance uses Freivalds' algorithm to verify the trustworthiness of the convolution computation and the correctness of its output C . The verification methodology IFAV will be discussed in Section IV in detail. If the IFAV finds that an attack occurs in the convolution computation, the TA instance sends a message that this inference case is attacked and the model output is incorrect. The result of the IFAV is transmitted from the TA instance to the subthread of the CA, and the subthread processes and outputs the detected attacks.

Therefore, DarkneTV could prevent attackers from modifying the convolution weight used for deep learning inference in the REE through the KHC in the TEE. In addition, it effectively detects the attacker's tampering with the inference process and results of time-intensive convolutions using the IFAV in the TEE. As a result, DarkneTV outputs correct or incorrect inference results with attack tips.

C. CPU–GPU Cooperative Execution

DarkneTV relies on the cooperative execution between multicore CPUs in the TEE and the integrated GPUs in the REE on heterogeneous edge devices. The idle CPU cores are used to detect the attack with the TA instance. At the same time, the integrated GPUs run normal operator inference in the REE to enhance the inference performance.

Three processes participate in the verifiable inference stage of DarkneTV, i.e., the CA and TA processes of the verifiable inference program and the daemon of OP-TEE named tee-supplciant. The CA process executes model operators with the integrated GPU and outputs inference and attack detection results. The TA process loads the encrypted model weight

files through the tee-supplciant and then decrypts them to obtain the weights of model operators. In particular, it also generates and checks hash signatures of convolution weight A and conducts the IFAV, aiming to detect attacks against A and the corresponding convolution computation. The daemon process tee-supplciant provides the synchronization and communication services between the TA instance and the CA, including reading encrypted weight files and transferring attack detection results to the CA.

The CA process of operator inference on the integrated GPU and the TA process of its attack detection in the CPU TEE are executed asynchronously. Therefore, the CA process does not need to block or suspend to wait for finishing KHC and IFVA. DarkneTV can fully use a powerful GPU (without TEE) to accelerate the model inference on edge devices. In this way, DarkneTV not only accelerates deep learning inference with the integrated GPU (without TEE) but also detects attacks against the convolution weights and inference process in the CPU TEE asynchronously. Then, it improves the performance of the verifiable deep learning inference on the heterogeneous edge device, which has a powerful GPU that does not support TEE.

Except for the above GPU-accelerated mode, DarkneTV also supports verifiable inference in the CPU-only mode. Their common is running both hash checking of convolution weights and convolution verification in the CPU TEE of the edge device. Their difference is that the latter executes all the operator inference on the CPU, incorporating KHC and IFAV, while the former performs the operator inference on the integrated GPU to enhance the inference performance.

D. Differences Analysis

The existing works [27], [28], [29], [30] deploy part layers of deep learning models to TEE to ensure the reliability of inference results. However, DarkneTZ [27], OMG [29], and Branchy-TEE [30] cannot effectively take advantage of the integrated GPUs in edge devices to accelerate the deep learning inference, while the convolution computations are typically time-consuming for CPUs on edge devices. SOTER [28] transforms partial model operators into parameter-morphed operators, uses the GPU to execute their inference, and then restores the computation results in the TEE. Its processes of operator transformations and results restoration require modifying the model operator and results. Thus, adversaries can also attack these processes, and SOTER increases the time overhead of deep learning inference as these processes. Due to the lack of support for GPU-accelerated inference or model operator transformations, these solutions have considerable inference latency on edge devices. To address this problem, DarkneTV directly deploys the inference process of all the model operators to the GPU in REE without any transformation for the trained model, while the convolution verification process is deployed to the TEE and executed with the introduced synchronous mechanism. Meanwhile, DarkneTV encrypts model weights and generates hash signatures for each convolution weight in the TEE offline before starting model inference to support hash checking of convolution weights in deep learning inference.

Furthermore, this work is mainly inspired by DarkneTZ [27], but they have significant differences listed as follows.

- 1) DarkneTZ deploys the last few continuous layers of the model as sensitive layers to TEE and cannot protect all the discontinuous and vulnerable model layers, e.g., convolutions in the former of the model. Unfortunately, falsifying the memory of convolution operators is the most effective way for FIAs since convolutions take more inference time than other model operators. This makes DarkneTZ easy to be attacked by modifying the former convolution layers in model inference. In contrast, DarkneTV flexibly selects discrete and time-consuming convolutions to be protected with a TEE-based verification mechanism and can enhance the reliability of the whole model inference process by efficiently defending against FIAs.
- 2) DarkneTZ cannot protect trained model weights stored on the edge device, whereas DarkneTV ensures the integrity and privacy of trained model weights by encrypting them with the secure storage functions of TEE in the model encryption stage.
- 3) DarkneTZ only supports model inference using the CPU, while DarkneTV supports CPU–GPU cooperative verifiable inference on the heterogeneous edge devices. Then, DarkneTV enables the model inference on the integrated GPU and the convolution verification in the CPU TEE to be executed asynchronously and effectively reduces the total time overhead of verifiable inference on heterogeneous edge devices.

In addition, the experimental results shown in Section VI further demonstrate the advantages of the proposed DarkneTV.

IV. VERIFY CONVOLUTION COMPUTATIONS IN TEE

A. Verify Convolutions With Freivalds' Algorithm

The im2col-based convolution algorithm [32] transforms convolution computations into general matrix multiplications. It leverages the efficient matrix multiplication routines to accelerate convolution computations. The convolution kernel and input feature map are flattened into the matrices $\mathbf{A}_{m \times k}$ and $\mathbf{B}_{k \times n}$, respectively. The output feature map is represented as the matrix $\mathbf{C}_{m \times n}$, and the matrix multiplication $\mathbf{A} \times \mathbf{B} = \mathbf{C}$ holds. Since convolutions are performed in the insecure REE, the matrix multiplication results should be verified to ensure their correctness. Each element in $\mathbf{C}$ requires k multiplications and $(k - 1)$ additions; the total number of operations is

$$k \times (m \times n) + (k - 1) \times (m \times n) = (2k - 1) \times mn.$$

Therefore, the time complexity of $O(n^3)$ is required to verify a convolution output by recalculating $\mathbf{A} \times \mathbf{B}$.

Inspired by the fact that Freivalds' algorithm [33] is an efficient matrix multiplication verification algorithm with time complexity $O(n^2)$. Freivalds' algorithm introduces a random column vector $\gamma_{n \times 1}$, $\gamma_i \in \{0, 1\}$ and verifies the assumption $\mathbf{A} \times (\mathbf{B} \times \gamma) = \mathbf{C} \times \gamma$. If this assumption is proved, the matrix multiplication $\mathbf{A} \times \mathbf{B}$ equals $\mathbf{C}$ with a high probability. This algorithm optimizes the matrix–matrix multiplication operation by transforming it into three matrix–vector multiplications. Using the example of multiplying the matrix

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix} \times \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix} \times \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$

$\mathbf{B}_1$ $\mathbf{B}_2$ $\mathbf{r}_1$
 $\mathbf{r}_2$

Fig. 4. Example of the partitioned matrix–vector multiplication.

$\mathbf{B}$ and the vector γ , this operation requires k multiplications and $k \times (n - 1)$ additions, which amounts to $k \times n$ operations. Thus, the total number of operations required by Freivalds' algorithm is $(k \times n + m \times k + n \times m)$, and its time complexity is $O(n^2)$. To reduce the time overhead of convolution verification, DarkneTV uses Freivalds' algorithm to verify the convolution computations.

Due to the vector γ containing zero elements, Freivalds' algorithm suffers from missing detection. For example, if the second element in the vector γ is zero, the variations in the second column elements of matrix $\mathbf{B}$ and $\mathbf{C}$ are ignored. In addition, the data type of convolution computations is the float. Therefore, we set the value of γ_i as floating-point numbers within the range of $(0.00, 1.00]$ to improve the detection rates and adapt to the data type of convolution computations

$$\epsilon_r = \|\mathbf{A} \times (\mathbf{B} \times \gamma) - \mathbf{C} \times \gamma\|_{\infty}. \quad (1)$$

Due to round-off errors of floating-point multiplications in computers, $\mathbf{A} \times (\mathbf{B} \times \gamma)$ may not be strictly equal to $\mathbf{C} \times \gamma$ for the IFAV. To address this problem, DarkneTV uses the infinity norm of compute errors ϵ_r [shown in (1)] to evaluate whether the convolution computation is attacked and outputs an incorrect $\mathbf{C}$.

$$\epsilon_r \leq 0.60 \times mk/2^{24}. \quad (2)$$

The upper limit of the round-off error arising from a floating-point multiplication in computers is 0.50×2^{-23} . Equation (2) should be held when the convolution computation is not attacked and outputs a correct $\mathbf{C}$. Otherwise, the convolution runtime is attacked, and the output result is wrong. In (2), 0.60 denotes an introduced probability coefficient for the IFAV.

B. Implementation Methodology on Edge Devices

Due to the security requirements of the TEE, the size of secure RAM in the TEE cannot be set too large. It is a challenge to perform the IFAV with the limited secure (random access memory) RAM of the TEE on edge devices. To address this problem, DarkneTV uses the partitioned matrix–vector multiplication to run the IFAV in the TEE. Taking the matrix multiplication shown in Fig. 4 as an example, the matrix $\mathbf{B}$ is divided into two submatrices ($\mathbf{B}_1$ and $\mathbf{B}_2$) by a vertical line, while the random vector γ is split into two subvectors (γ_1 and γ_2) by a horizontal line. Then, $\mathbf{B} \times \gamma$ performs by considering the submatrices individually as the following equation:

$$\mathbf{B} \times \gamma = [\mathbf{B}_1 \ \mathbf{B}_2] \times \begin{bmatrix} \gamma_1 \\ \gamma_2 \end{bmatrix} = [\mathbf{B}_1 \times \gamma_1 + \mathbf{B}_2 \times \gamma_2]. \quad (3)$$

Thus, DarkneTV first calculates the matrix–vector multiplication $\mathbf{B}_1 \times \gamma_1$ within a TA instance. Then, it performs the other

matrix–vector multiplication $\mathbf{B}_2 \times \gamma_2$ in the same TA instance and adds their intermediate vectors stored in the TEE secure memory to obtain the output of $\mathbf{B} \times \gamma$. Thus, it reduces the requirement of secure memory in the TEE for performing the whole $\mathbf{B} \times \gamma$. This is because it transfers a smaller submatrix $\mathbf{B}_1$ or $\mathbf{B}_2$ to the TA instance and performs two matrix–vector multiplications separately instead of transferring the entire matrix $\mathbf{B}$ to the TA instance and performing the multiplication of a large matrix and the vector γ at once.

Similarly, the matrix–vector multiplications $\mathbf{C} \times \gamma$ and $\mathbf{A} \times \gamma_{\mathbf{B}}$ ($\gamma_{\mathbf{B}} = \mathbf{B} \times \gamma$) are implemented with the similar method shown in (3). Thus, the verification of large convolutions can be implemented in the TEE with limited secure RAM on edge devices.

V. THREAT MODEL AND SIMULATION

The premise of this work assumes that trained deep learning models are deployed to edge devices without trojans and backdoors [34], [35]. It also presumes that adversaries are mainly interested in destroying deep learning inference on edge devices through FIAs. Zhao et al. [12] suggest that the ℓ_2 norm is suitable for measuring the modification magnitude of FIAs against deep learning inference. We also use the ℓ_2 norm to evaluate the modification magnitude applied to each modified element in the attacked matrix. In addition, Zhao et al. [12] measure the number of modifications with the ℓ_0 norm to indicate the extent of an FIA. However, the more data involved in an operator, the more data must be modified if the adversaries want to tamper with the results of the model inference. Therefore, the proportion of modifications p_m is more scientific than the number of modifications for evaluating the extent of FIAs.

We simulate FIAs by randomly modifying the convolution weights and outputs to implement the above thread model in this work. The simulated FIAs randomly select attacked locations and control the extent and magnitude of modifications by setting modification proportion p_m and modification magnitude ℓ_2 norm.

Algorithm 1 Simulated FIAs

Input: Target matrix $\mathbf{T}_{m \times n}$, modification proportion p_m , and modification magnitude ℓ_2 norm.

Output: No output

```

1:  $x \leftarrow \text{int}(m \times n \times p_m)$ 
2: if  $x == 0$  then
3:    $x \leftarrow 1$ 
4: end if
5:  $e \leftarrow \sqrt{(\ell_2 \text{ norm})^2 / x}$ 
6: for  $i = 0$  to  $x$  do
7:    $a \leftarrow \text{non\_repeat\_rand}(m \times n)$ 
8:   if  $\text{rand}() \% 2 == 1$  then
9:      $T_a \leftarrow T_a + e$ 
10:  else
11:     $T_a \leftarrow T_a - e$ 
12:  end if
13: end for
```

TABLE I
EXPERIMENTAL ENVIRONMENT WITH THE HIKEY 960

Component	Description
CPU	4 Cortex A73@2.36GHZ + 4 Cortex A53@1.84GHZ
GPU	ARM Mali-G71 MP8@1.04GHZ
RAM	4GB LPDDR4 SDRAM
Storage	32GB SD card
OS	Debian bullseye with kernel 4.19
OpenCL	Open Computing Language 1.2
OP-TEE	OP-TEE 3.16.0

We design a simulated FIA algorithm to simulate the FIAs on deep learning inference. The pseudocode of the simulated FIA algorithm is shown as Algorithm 1. The input of Algorithm 1 is the target matrix $\mathbf{T}_{m \times n}$ and the modification proportion and magnitude (p_m and ℓ_2 norm) of the simulated FIA. Lines 1–5 are to obtain the modification number x of matrix elements and the modification magnitude e of each matrix element. In Line 7, the function `non_repeat_rand( $m \times n$ )` returns a random integer a to locate the matrix element to be modified by the simulated FIA, where a is in the range of $[0, (m \times n) - 1]$ and does not repeat any previously returned values. Namely, T_a denotes the a th elements in the target matrix $\mathbf{T}_{m \times n}$. Lines 8–12 modify the target matrix elements based on the parity property of the random number a and the modification magnitude e .

VI. EXPERIMENTAL ENVIRONMENT AND RESULTS

A. Experimental Environment

DarknetTV is implemented by introducing the TEE-based model encryption and verifiable inference shown in Section III into the Darknet framework [8], which is a deep learning library programming with C and OpenCL. OpenCL [15] is a parallel programming standard for writing portable code that executes across heterogeneous platforms, including CPUs, GPUs, and so on. It can be adopted to accelerate deep learning inference with the integrated GPUs on heterogeneous edge devices. Meanwhile, the open portable TEE framework OP-TEE [25] enables these heterogeneous edge devices to perform TEE-based model encryption and verifiable inference with TEEs. Therefore, the proposed DarknetTV is valid for different heterogeneous edge devices that consist of CPUs with TEE and GPUs with OpenCL support.

We experiment with the Hikey 960 development platform as a heterogeneous edge device. Table I illustrates the experimental environment constructed with the HiKey 960, including its hardware and software characteristics. The Hikey 960 is equipped with an octa-core ARM big.LITTLE processor with four ARM Cortex-A73@2.36GHZ and four Cortex-A53@1.84GHZ cores, and a Mali G71 MP8 GPU@1.04GHZ GPU with eight shader cores. Its CPU supports OP-TEE, and its integrated GPU supports OpenCL-enabled parallel computation, enabling it to satisfy the experiment requirements in this work. Besides, Debian 11 (Bullseye) is installed on the Hikey 960, and OpenCL 1.2 and OP-TEE 3.16.0 are used to implement the proposed DarknetTV. The Hikey 960 used in the experiment has a total memory size of 4 GB for REE. To support the verifiable inference of large convolution

operators, the secure memory size of TEE is set to 64 MB, with 60 MB allocated for running TAs and 4 MB dedicated to the TEE runtime.

Image classification and object detection are the two representative subfields of deep learning applications. Then, we leverage two image classification models (Darknet19 [36] and Resnet18 [9]) and two object detection models (YOLOv3-tiny [37] and YOLOv4-tiny [10]) as example models to evaluate the proposed DarkneTV.

B. Evaluation Methods

To demonstrate the effects of FIAs against deep learning inference, we adopt Algorithm 1, described in Section V, to simulate the model inference with fault disturbance by randomly modifying the convolution weight $\mathbf{A}$ and its output $\mathbf{C}$ when conducting the deep learning inference. Then, we evaluate DarkneTV's performance using two metrics: the effectiveness of the FIA detection and the performance of verifiable inference on heterogeneous edge devices.

The effectiveness of the FIA detection is gauged by the attack detection rate achieved by the proposed DarkneTV. The higher the attack detection rate, the better DarkneTV's defense against FIAs.

To evaluate the performance of verifiable inference implemented with DarkneTV on heterogeneous edge devices, its inference speed, extra time proportion, time reduction, and speedups of CPU–GPU collaboration are measured and compared with normal inference. The normal inference means executing model inference on either CPU or GPU without attack detection (KHC and IFV) and is implemented with the Darknet [8]. In contrast, verifiable inference means executing model inference on either CPU or CPU+GPU with the TEE-based attack detection implemented with the proposed DarkneTV. The inference speed is measured with the average latency in seconds for generating a model output (its unit is s/image). The extra time in seconds is taken by detecting attacks in the verifiable inference compared with its counterpart normal inference. In addition, the time reduction and speedup of GPU-accelerated verifiable inference are measured and compared with those using only the CPU. This aims to demonstrate performance improvement as the CPU–GPU cooperative execution is introduced in the proposed DarkneTV.

We use the images selected from the ImageNet-1K [38] and COCO [39] test datasets to measure DarkneTV's inference performance. We execute model inference consecutively at a time with all the selected images. For each evaluation metric, we repeat its measurement five times and use the average value as the experimental result implemented with DarkneTV.

C. Attacked Inference Cases

We simulate FIAs to modify the output feature $\mathbf{C}$ of each convolutional layer in the compared models. Figs. 5 and 6 show the attack results compared with the corresponding results without FIAs.

Fig. 5 suggests that there are only tiny confidence changes in the major categories for image classification of Darknet19 and Resnet18 with different ℓ_2 norm and p_m combinations.

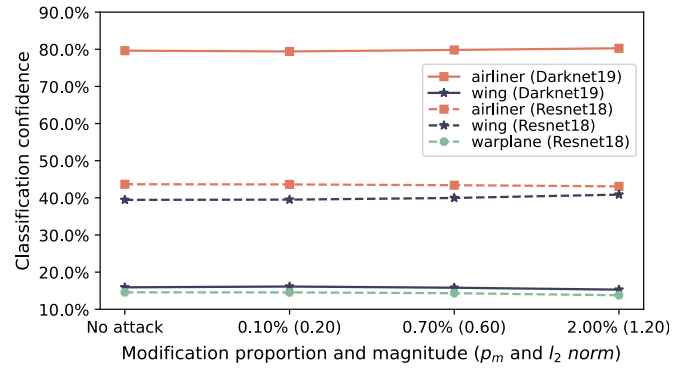


Fig. 5. Classification results of Darknet19 and Resnet18 when $\mathbf{C}$ is attacked.

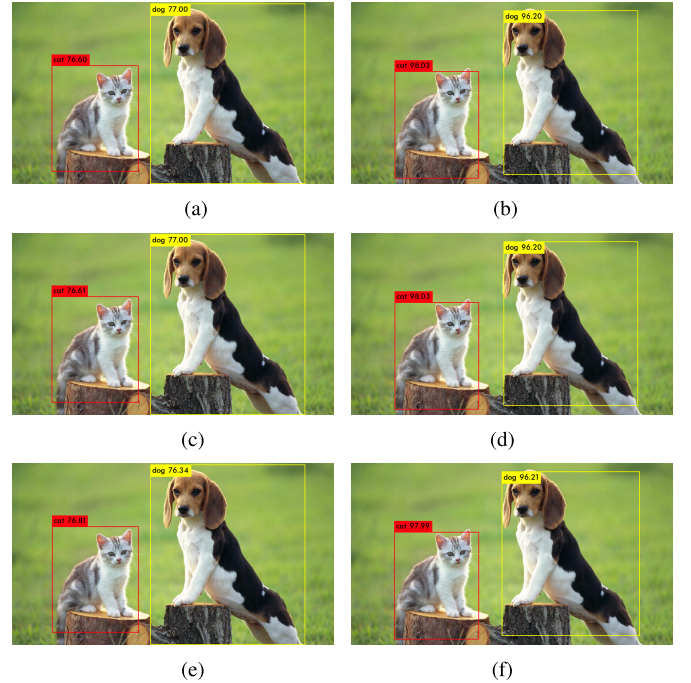


Fig. 6. Perturbed object detection results with different attacks on $\mathbf{C}$. (a) YOLOv3-tiny W/o attack. (b) YOLOv4-tiny W/o attack. (c) W/ attack $\mathbf{C}$ 0.20% (0.10). (d) W/ attack $\mathbf{C}$ 0.20% (0.10). (e) W/ attack $\mathbf{C}$ 2.00% (1.50). (f) W/ attack $\mathbf{C}$ 2.00% (0.80).

For Darknet19, the classification confidence of “wing” slightly decreased, whereas that of “airliner” shows an upward trend when the attack's p_m and ℓ_2 norm increase from 0.00% and 0.00 (No attack) to 2.00% and 1.20. Regarding the inference results of Resnet18, comparing the “No attack” and the attack with $p_m = 2.00\%$ and ℓ_2 norm = 1.20, the classification confidence variations for “airliner,” “wing,” and “warplane” are -0.57% , $+1.42\%$, and -0.79% , respectively.

In Fig. 6, the first row shows the results without attacks against YOLOv3-tiny and YOLOv4-tiny, which are used as the baselines for comparing with the attacked inference results. The remaining rows show the perturbed object detection results with the simulated attacks, where the percentage (0.20%, 2.00%) is the modification proportion p_m , and the floats (0.10, 0.80, 1.50) in parentheses represent the values of the modification magnitude ℓ_2 norm. For the attack with $p_m = 0.20\%$ and ℓ_2 norm = 0.10 on object detection,

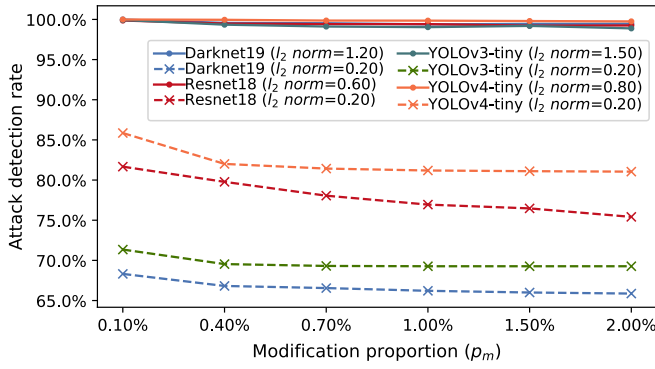


Fig. 7. Attack detection rates for attacking $\mathbf{C}$.

the confidence and box locations of the objects are nearly unchanged, as shown in Fig. 6(c) and (d). Similarly, the box locations and number of the detected objects are also not changed, but a tiny falsification in confidence of the objects when both p_m and ℓ_2 norm are increased. For the object detection with YOLOv3-tiny and the attack ($p_m = 2.00\%$, ℓ_2 norm = 1.50), the confidence of the “cat” increases by 0.21, whereas the confidence of the “dog” decreases by 0.66, as shown in Fig. 6(e). For YOLOv4-tiny, under the attack ($p_m = 2.00\%$, ℓ_2 norm = 0.80), the confidence of “cat” and “dog” decreases by 0.04 and 0.01, respectively, as shown in Fig. 6(f). Similar to Fig. 5, Fig. 6 suggests that there is only a tiny variation in the confidence of perturbed object detection with different FIAs on the convolution output $\mathbf{C}$.

D. Effectiveness of Attack Detection

As the SHA-256 algorithm operates at a bit level, it remains unbroken until now, with no known methods for effectively cracking it. Thus, the SHA-256 algorithm is highly sensitive to even the slightest alterations in data. Thus, any number and magnitude of modifications in the convolution weight $\mathbf{A}$ can be detected by DarkneTV with 100.00% accuracy due to the sensitivity of hash checking based on the SHA-256 algorithm. Therefore, this section mainly analyzes the attack detection rates for the convolution output $\mathbf{C}$ at different alterations.

We simulate FIAs against $\mathbf{C}$ in each convolution operator at $p_m \in \{0.10\%, 0.40\%, 0.70\%, 1.00\%, 1.50\%, 2.00\%\}$ and test the attack detection rates for four example models. For each modification proportion, the attack detection rates at the modification magnitude ℓ_2 norm = 0.20 are used as the comparison baselines, denoted as the dotted lines in Fig. 7. The attacks with ℓ_2 norm = 0.20 make the attack detection rates larger than 65.87% for four models at any p_m . The attack detection rate achieves the highest for YOLOv4-tiny, exceeding 81.05% at all the tested p_m when ℓ_2 norm = 0.20. In contrast, it exhibits the lowest one when ℓ_2 norm = 0.20, remaining below 68.32% for Darknet19. Then, we simulate FIAs starting with the ℓ_2 norm = 1.20 and changing it by ± 0.10 to find its minimum value that enables nearly all the attacks detected (detection rate exceeds 99.00%) for the given deep learning model. These minimum values of the ℓ_2 norm are denoted with the solid lines in Fig. 7. When the attack

detection rate exceeds 99.00%, the ℓ_2 norm reaches 0.60, 0.80, 1.20, and 1.50 for the models Resnet18, YOLOv4-tiny, Darknet19, and YOLOv3-tiny, respectively,

$$\mathbf{C} = \begin{bmatrix} 1.00 & 2.00 & \cdots & 3.00 \\ 2.00 & 4.00 & \cdots & 7.00 \\ \cdots & \cdots & \cdots & \cdots \\ 3.00 & 6.00 & \cdots & 5.00 \end{bmatrix}_{20 \times 20} \quad (4)$$

$$\mathbf{C}_1 = \begin{bmatrix} 1.00 & \underline{2.50} & \cdots & 3.00 \\ 2.00 & \underline{3.50} & \cdots & 7.00 \\ \cdots & \cdots & \cdots & \cdots \\ 3.00 & 6.00 & \cdots & 5.00 \end{bmatrix}_{20 \times 20} \quad (5)$$

$$\mathbf{C}_2 = \begin{bmatrix} \underline{0.75} & 2.00 & \cdots & 3.00 \\ 2.00 & \underline{3.75} & \cdots & \underline{7.25} \\ \cdots & \cdots & \cdots & \cdots \\ \underline{3.25} & 6.00 & \cdots & 5.00 \end{bmatrix}_{20 \times 20} \quad (6)$$

For all the compared models, the attack detection rates at $p_m = 0.10\%$ are higher than the cases when p_m increases to 2.00%. This is because the ℓ_2 norm represents the modification magnitude for the entire attacked $\mathbf{C}$, and the average modification magnitude for each modified element decreases when the modification proportion increases. When a convolution outputs the matrix $\mathbf{C}$ size of 20×20 shown in (4), its output can be changed into $\mathbf{C}_1$ and $\mathbf{C}_2$ through the attacks with the same ℓ_2 norm = 1.00. In the attacked $\mathbf{C}_1$ and $\mathbf{C}_2$, the attacked elements are identified by underlining.

When $p_m = 0.50\%$, the number of modified elements for $\mathbf{C}$ is $0.50\% \times 20 \times 20 = 2.00$. The variation in each modified element is $1.00 / 2.00 = 0.50$. In this case, the attacked $\mathbf{C}$ is changed to $\mathbf{C}_1$, as shown in (5). The elements 2.00 and 4.00 are modified into 2.50 and 3.50, respectively. In contrast, when p_m increases to 1.00%, the number of modified elements increases to $1.00\% \times 20 \times 20 = 4.00$. The variation in each modified element is $1.00 / 4.00 = 0.25$. The attacked $\mathbf{C}$ is changed into $\mathbf{C}_2$, shown in (6), where the elements 1.00, 4.00, 7.00, and 3.00 are modified through -0.25 , -0.25 , $+0.25$, and $+0.25$. When the variation in each modified element is slight enough, it is more likely to be considered a floating-point arithmetic error. Therefore, the attack detection rates fall slowly when p_m increases, but the ℓ_2 norm remains the same.

In addition, Fig. 7 suggests that DarkneTV could provide more than 99.03% detection accuracy for FIAs against the convolution computation when ℓ_2 norm ≥ 1.50 and $p_m \leq 2.00\%$. According to [11] and [12], to successfully modify the output result of a deep learning model, the ℓ_2 norm of the modified convolution weight $\mathbf{A}$ requires to be more than 10.00. This means that the ℓ_2 norm of the tampered operator output $\mathbf{C}$ also requires to be more than 10.00. Then, we also simulate the FIAs against the convolution output $\mathbf{C}$ with ℓ_2 norm = 10.00 and $p_m \leq 2.00\%$ to measure the attack detection rates of the four example models. The results suggest that DarkneTV could detect these FIAs with 100.00% accuracy for convolution computations. Consequently, DarkneTV could defend against FIAs with more than 99.03% detection accuracy for both tiny perturbations and significant modifications in convolution computations.

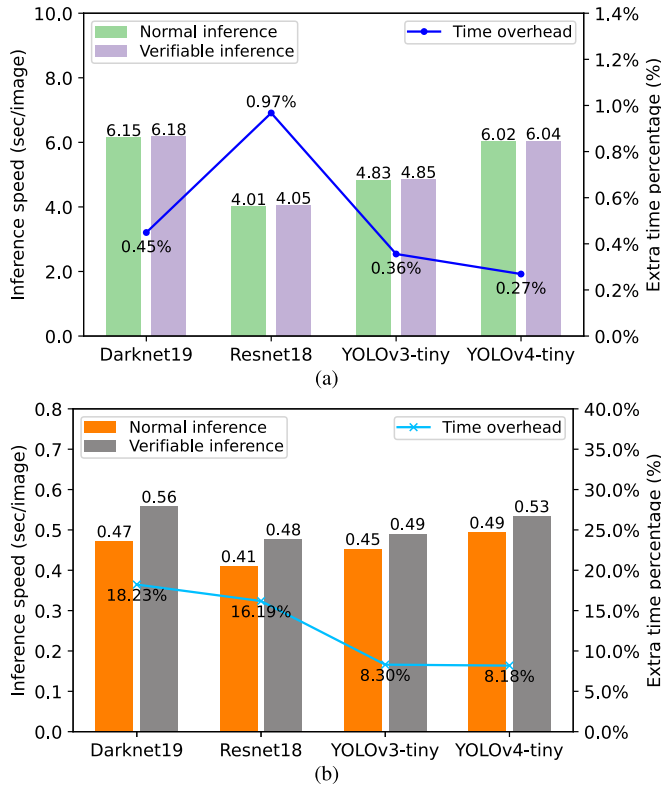


Fig. 8. Comparison on the speed and the percentage of extra time as verifiable inference. (a) Inference with CPU only. (b) Inference accelerated with GPU.

E. Inference Performance

1) *Inference Speed and Extra Time:* To evaluate the speed and extra time overhead of verifiable inference implemented with DarkneTV, we compare it with the normal inference implemented using the OpenCL-enabled Darknet [8]. Fig. 8(a) shows comparison of the speed of normal and verifiable inference only using the CPU on the Hikey 960. Fig. 8(b) shows comparison of the speed of these two types of inference accelerated with the OpenCL-enabled GPU on the Hikey 960. They also show the proportion of extra time on average arising from the verifiable inference compared with the corresponding normal inference.

Fig. 8(a) shows that the four models' maximum speed of verifiable inference on the CPU achieves 4.05 s/image (Resnet18), whereas the minimum speed is 6.18 s/image (Darknet19). The extra time taken by attack detection in the verifiable inference is about 0.02–0.04 seconds per image for the four deep learning models, compared with their normal inference time implemented with the Darknet on the CPU. Correspondingly, their extra time percentage is not more than 0.97% for the verifiable inference with the CPU-only mode. The lowest proportion is about 0.27%, taken by the verifiable inference of YOLOv4-tiny on the CPU. This suggests that the verifiable inference in the CPU-only mode can identify FIAs with a considerably small time overhead and nearly not increase the time overhead. This is since the operator inference and its attack detection run on different CPU cores using independent threads asynchronously.

Fig. 8(b) demonstrates that the speed of GPU-accelerated verifiable inference significantly improved for the

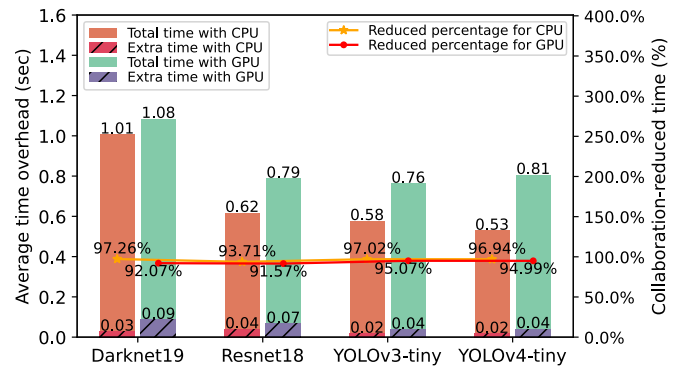


Fig. 9. Time reduction of attack detection with asynchronous collaboration.

compared deep learning models. For instance, the speed of GPU-accelerated verifiable inference of Resnet18 reaches up to 0.48 s/image and improves about 8.50times. Similarly, the speed of GPU-accelerated verifiable inference of Darknet19 increases from 6.18 to 0.56 s/image. Unfortunately, the four models' extra time of GPU-accelerated verifiable inference with DarkneTV increases to 0.04–0.09 s per image. Accordingly, the percentage of the extra time per image taken by the attack detection grows up to 8.18%–18.23% when accelerating verifiable inference with the integrated GPU on the edge device. It is approximately 17.76–30.30× more than the verification inference in the CPU-only mode. This is because the time of an operator computation with the powerful GPU is sharply reduced compared with the low-performance CPU. Meanwhile, the extra time taken by convolutions' attack detection is not reduced but increased when moving from the CPU-based model inference to the OpenCL-enabled model inference on the integrated GPU. The increased extra time mainly arises from reading and transferring data (A and C) from the integrated GPU to the CPU on the edge devices through the OpenCL buffer.

2) *Time Reduction of Asynchronous Collaboration:* Fig. 9 shows comparison of the average time taken by the attack detection of all the convolutions with asynchronous collaboration in both the CPU-only and GPU-accelerated modes. The total time measures the sum of time taken by detecting attacks in the TEE for each convolution in the given model, while the extra time is the actual time taken by the attack detection during the verifiable inference of the given model.

Fig. 9 indicates that the actual extra time of verifiable inference is significantly reduced using the asynchronous collaboration that computes the operators in the REE with CPU or GPU while detecting attacks in the CPU TEE. In the CPU-only mode, the total time of attack detection reaches up to 0.53–1.01 s for all the convolutions of a deep learning model when running a verifiable inference. However, the actual extra time overhead is only 0.02–0.04 s for the proposed DarkneTV. In contrast, the extra time the attack detection takes in the verifiable inference of one model is reduced by 93.71%–97.26%. For instance, the extra time taken by the verifiable inference of YOLOv3-tiny and YOLOv4-tiny on the CPU is reduced to 0.02 s for each convolution's input.

Similarly, the extra time taken by the attack detection in the GPU-accelerated mode also reduces by 91.57%–95.07%.

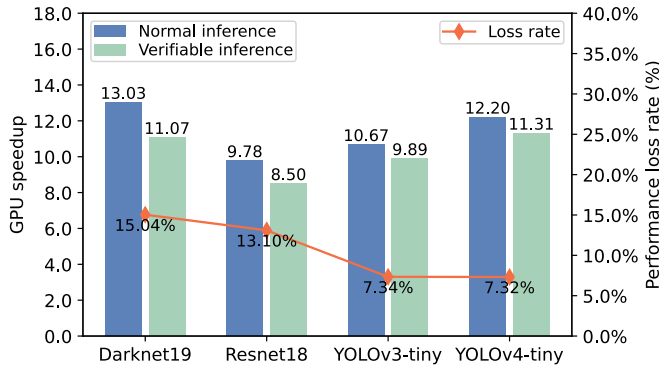


Fig. 10. Speedup comparisons of GPU-accelerated model inference.

The reduced extra time for Resnet18 is 0.72 s compared with the total time taken by the attack detection for all the convolutions in this model. Unfortunately, the extra time reduced in the GPU-accelerated mode is less than the CPU-only mode. This is due to reading the convolution weight **A** and outputting **C** from GPU-based OpenCL buffers to CPU memory before transferring them to TEE. However, the extra time for verifiable inference in both the CPU-only and GPU-accelerated modes is significantly reduced using asynchronous collaboration of attack detection and operator computation on different compute units.

3) Speedup Comparison: Fig. 10 shows comparison of the speedups of GPU-accelerated verifiable and normal inference in terms of their counterparts (CPU-only model). Meanwhile, it also shows the performance loss rate, which is the percentage of lost speedup when moving from the normal inference to the verifiable inference. The loss rate is calculated with the following equation:

$$\text{loss rate} = (s_n - s_v)/s_n \times 100.00\% \quad (7)$$

where s_n denotes the speedup of the GPU-accelerated normal inference, and s_v denotes the speedup of the GPU-accelerated verifiable inference.

As shown in Fig. 10, the maximum and minimum speedups of the GPU-accelerated verifiable inference on the Hikey 960 achieve 11.31 $\times$ (YOLOv4-tiny) and 8.50 $\times$ (Resnet18), respectively, compared with the speed of their verifiable inference on the CPU-only mode. The speedup of the GPU-accelerated normal inference with the OpenCL-enabled Darknet [8] is 9.78 $\times$ –13.03 $\times$ compared with the models' normal inference only using the CPU. Accordingly, the speedups for GPU-accelerated verifiable inference only have a minimum loss rate of 7.32%–15.04% compared with the speedups of GPU-accelerated normal inference obtained by the OpenCL-enabled Darknet. Compared with the obtained significant speedup and the corresponding normal inference implemented with OpenCL-enabled Darknet on the same GPU, the performance loss is trivial due to the attack detection function introduced by the GPU-enabled DarkneTV. Therefore, DarkneTV significantly improves the performance of verifiable inference on heterogeneous edge devices, similar to the normal inference on OpenCL-enabled edge devices.

VII. CONCLUSION

DarkneTV is a verifiable and efficient deep learning inference solution using the CPU TEE for heterogeneous edge devices. It introduces a verification scheme based on the TEE, effectively detecting FIAs on convolution weights and their computations during model inference. It performs GPU-accelerated operator inference and TEE-based attack detection asynchronously, significantly accelerating verifiable inference using the integrated GPU on heterogeneous edge devices. The GPU-accelerated verifiable inference obtains a similar performance as the normal inference with the OpenCL-enabled Darknet [8] on the Hikey 960. Its speed is improved by 8.50 $\times$ –11.31 $\times$ compared with the counterpart in the CPU-only mode.

REFERENCES

- [1] G. Nain, K. K. Pattanaik, and G. K. Sharma, "Towards edge computing in intelligent manufacturing: Past, present and future," *J. Manuf. Syst.*, vol. 62, pp. 588–611, Jan. 2022.
- [2] M. Alkhalid, M. B. Taha, R. Chowdhury, H. Ould-Slimane, A. Mourad, and C. Talhi, "Vehicular edge-based approach for optimizing urban data privacy," *IEEE Sensors J.*, vol. 24, no. 5, pp. 5609–5621, Mar. 2024.
- [3] L. Zhu, Z. Zhang, L. Liu, L. Feng, P. Lin, and Y. Zhang, "Online distributed learning-based load-aware heterogeneous vehicular edge computing," *IEEE Sens. J.*, vol. 23, no. 15, pp. 17350–17365, Aug. 2023.
- [4] S. Wang, I. Li, and W. Wang, "Human action recognition of autonomous mobile robot using edge-AI," *IEEE Sensors J.*, vol. 23, no. 2, pp. 1671–1682, Jan. 2023.
- [5] B. Yuce, P. Schaumont, and M. Witteman, "Fault attacks on secure embedded software: Threats, design, and evaluation," *J. Hardw. Syst. Secur.*, vol. 2, no. 2, pp. 111–130, Jun. 2018.
- [6] G. Liang, S. R. Weller, J. Zhao, F. Luo, and Z. Y. Dong, "The 2015 Ukraine blackout: Implications for false data injection attacks," *IEEE Trans. Power Syst.*, vol. 32, no. 4, pp. 3317–3318, Jul. 2017.
- [7] E. Naderi and A. Asrari, "A deep learning framework to identify remedial action schemes against false data injection cyberattacks targeting smart power systems," *IEEE Trans. Ind. Informat.*, vol. 20, no. 2, pp. 1208–1219, Feb. 2024.
- [8] P. Sowa and J. Izydorczyk, "Darknet on OpenCL: A multiplatform tool for object detection and classification," *Concurrency Comput., Pract. Exper.*, vol. 34, no. 15, p. e6936, Jul. 2022.
- [9] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [10] A. Bochkovskiy, C.-Y. Wang, and H.-Y. Mark Liao, "YOLOv4: Optimal speed and accuracy of object detection," 2020, *arXiv:2004.10934*.
- [11] Y. Liu, L. Wei, B. Luo, and Q. Xu, "Fault injection attack on deep neural network," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2017, pp. 131–138.
- [12] P. Zhao, S. Wang, C. Gongye, Y. Wang, Y. Fei, and X. Lin, "Fault sneaking attack: A stealthy framework for misleading deep neural networks," in *Proc. 56th ACM/IEEE Design Autom. Conf. (DAC)*, Jun. 2019, pp. 1–6.
- [13] J. Bai, B. Wu, Y. Zhang, Y. Li, Z. Li, and S.-T. Xia, "Targeted attack against deep neural networks via flipping limited weight bits," 2021, *arXiv:2102.10496*.
- [14] ARM Security Technology Building a Secure System Using TrustZone Technology, ARM Ltd., Cambridge, U.K., 2019.
- [15] P. Heinisch, K. Ostaszewski, and H. Ranocha, "Towards green computing: A survey of performance and energy efficiency of different platforms using OpenCL," in *Proc. Int. Workshop OpenCL*, New York, NY, USA: Association for Computing Machinery, Apr. 2020, pp. 1–2.
- [16] Y. Kim, "Flipping bits in memory without accessing them: An experimental study of DRAM disturbance errors," *ACM SIGARCH Comput. Archit. News*, vol. 42, no. 3, pp. 361–372, Jun. 2014.
- [17] B. Selmk, S. Brummer, J. Heyzl, and G. Sigl, "Precise laser fault injections into 90 nm and 45 nm SRAM-cells," in *Proc. Int. Conf. Smart Card Res. Adv. Appl.*, 2015, pp. 193–205.
- [18] N. Moro, A. Dehbaoui, K. Heydemann, B. Robisson, and E. Encrenaz, "Electromagnetic fault injection: Towards a fault model on a 32-bit microcontroller," in *Proc. Workshop Fault Diagnosis Tolerance Cryptogr.*, Aug. 2013, pp. 77–88.

- [19] V. Khuat, J.-L. Danger, and J.-M. Dutertre, "Laser fault injection in a 32-bit microcontroller: From the flash interface to the execution pipeline," in *Proc. Workshop Fault Detection Tolerance Cryptography (FDTC)*, Sep. 2021, pp. 74–85.
- [20] J. Breier, X. Hou, D. Jap, L. Ma, S. Bhasin, and Y. Liu, "Practical fault attack on deep neural networks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 2204–2206.
- [21] S. Boyd et al., "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Found. Trends Mach. Learn.*, vol. 3, no. 1, pp. 1–122, 2011.
- [22] F. Yao, A. S. Rakin, and D. Fan, "DeepHammer: Depleting the intelligence of deep neural networks through targeted chain of bit flips," in *Proc. USENIX Secur. Symp.*, 2020, pp. 1463–1480.
- [23] W. Li, Y. Xia, and H. Chen, "Research on ARM TrustZone," *GetMobile, Mobile Comput. Commun.*, vol. 22, no. 3, pp. 17–22, Jan. 2019.
- [24] S. Pinto and N. Santos, "Demystifying arm trustzone: A comprehensive survey," *ACM Comput. Surveys*, vol. 51, no. 6, pp. 1–36, 2019.
- [25] L-Limited. (2014). *OP-TEE OS*. [Online]. Available: https://github.com/OP-TEE/optee_os
- [26] L-Limited. (2024). *Trusted Firmware-A*. [Online]. Available: <https://www.trustedfirmware.org/projects/tf-a/>
- [27] F. Mo et al., "DarkneTZ: Towards model privacy at the edge using trusted execution environments," in *Proc. 18th Int. Conf. Mobile Syst., Appl., Services*, Jun. 2020, pp. 161–174.
- [28] T. Shen et al., "SOTER: Guarding black-box inference for general neural networks at the edge," in *Proc. USENIX Annu. Tech. Conf.*, 2022, pp. 723–738.
- [29] S. P. Bayerl et al., "Offline model guard: Secure and private ML on mobile devices," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2020, pp. 460–465.
- [30] Y. Wang, K. Deng, F. Meng, Z. Chen, M. Yin, and R. Yang, "Branchy-TEE: Deep learning security inference acceleration using trusted execution environment," in *Proc. Int. Conferences Softw. Eng. Knowl. Eng.*, Jul. 2023, pp. 499–504.
- [31] E. Barker et al., *Secure Hash Standard (SHS)*, Standard FIPS 180-4, Federal Information Processing Standards (FIPS) Publications (PUBS), Aug. 2015.
- [32] H. Wang and C. Ma, "An optimization of im2col, an important method of CNNs, based on continuous address access," in *Proc. IEEE Int. Conf. Consum. Electron. Comput. Eng. (ICCECE)*, Jan. 2021, pp. 314–320.
- [33] R. Freivalds, "Probabilistic machines can use less running time," in *Proc. IFIP Congr.*, vol. 839, 1977, p. 842.
- [34] T. Gu, B. Dolan-Gavitt, and S. Garg, "BadNets: Identifying vulnerabilities in the machine learning model supply chain," 2017, *arXiv:1708.06733*.
- [35] B. Zhao and Y. Lao, "Towards class-oriented poisoning attacks against neural networks," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2022, pp. 2244–2253.
- [36] J. Redmon and A. Farhadi, "YOLO9000: Better, faster, stronger," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 7263–7271.
- [37] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," 2018, *arXiv:1804.02767*.
- [38] O. Russakovsky et al., "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, Dec. 2015.
- [39] X. Chen et al., "Microsoft COCO captions: Data collection and evaluation server," 2015, *arXiv:1504.00325*.



Yuqiang Zheng is currently pursuing the master's degree with the College of Computer and Data Science, Fuzhou University, Fuzhou, China.

His current research interests include edge learning, system security, and object detection.



Hong Lu received the Ph.D. degree in computer science from Nanjing University, Nanjing, China, in 2023.

She is an Associate Professor with Nanjing Normal University of Special Education, Nanjing. Her research interests include deep learning and intelligent construction.



Xinqi Liu received the M.Sc. degree from Hong Kong University of Science and Technology, Hong Kong, in 2021. He is pursuing the Ph.D. degree with the Department of Civil Engineering, University of Hong Kong, Hong Kong.

His research interests include edge learning and intelligent construction.



Shuguang Chen received the Ph.D. degree from the University of Hong Kong, Hong Kong.

He is a Research Assistant Professor with the University of Hong Kong, and a Managing Director of Hong Kong Quantum AI Laboratory, Hong Kong. His research interest is intelligent construction.



Longlong Liao received the Ph.D. degree from the National University of Defense Technology, Changsha, China, in 2019.

He is a Master Supervisor with Fuzhou University, Fuzhou, China, and a Post-Doctoral Fellow with The University of Hong Kong, Hong Kong. His research interests are edge intelligence and computer vision.



Yuanlong Yu received the Ph.D. degree in computer engineering from the Memorial University of Newfoundland, St. John's, NL, Canada, in 2010.

He is the Dean with the College of Computer and Data Science, Fuzhou University, Fuzhou, China. His research interests include machine learning, computer vision, and cognitive robotics.